

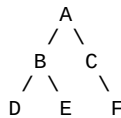
Teaching Notes: Trees (with C Programs)

2.1 Introduction to Trees

A Tree is a non-linear data structure that represents data in a hierarchical manner. It consists of nodes connected by edges.

Key Terms: Root, Parent, Child, Leaf, Edge, Height.

Example:



Properties:

1. A tree with n nodes has $n-1$ edges.
2. The root has no parent.
3. Each node except root has exactly one parent.

C Program: Basic Tree Creation

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main() {
    struct Node* root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);

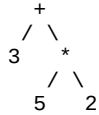
    printf("Inorder Traversal: ");
    inorder(root);
    return 0;
}
```

Output: Inorder Traversal: 4 2 5 1 3

2.2 Expression Trees

An Expression Tree is a binary tree where leaves are operands and internal nodes are operators.

Representation of Arithmetic Expressions



Construction of Expression Tree from Postfix

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

struct Node {
    char data;
    struct Node *left, *right;
};

struct Node* createNode(char data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = data;
    node->left = node->right = NULL;
    return node;
};

struct Node* stack[50];
int top = -1;

void push(struct Node* node) { stack[++top] = node; }
struct Node* pop() { return stack[top--]; }

struct Node* constructTree(char postfix[]) {
    for (int i = 0; postfix[i] != '\0'; i++) {
        char ch = postfix[i];
        if (isdigit(ch)) push(createNode(ch));
        else {
            struct Node* node = createNode(ch);
            node->right = pop();
            node->left = pop();
            push(node);
        }
    }
    return pop();
}

void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%c ", root->data);
        inorder(root->right);
    }
}

int main() {
    char postfix[] = "ab+ef*g*-";
    struct Node* root = constructTree(postfix);

    printf("Inorder Traversal of Expression Tree: ");
    inorder(root);
    return 0;
}
```

Evaluation of Expression Tree

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

struct Node {
    char data;
    struct Node *left, *right;
};

struct Node* createNode(char data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = data;
    node->left = node->right = NULL;
    return node;
}

struct Node* stack[50];
int top = -1;

void push(struct Node* node) { stack[++top] = node; }
struct Node* pop() { return stack[top--]; }

struct Node* constructTree(char postfix[]) {
    for (int i = 0; postfix[i] != '\0'; i++) {
        char ch = postfix[i];
        if (isdigit(ch)) push(createNode(ch));
        else {
            struct Node* node = createNode(ch);
            node->right = pop();
            node->left = pop();
            push(node);
        }
    }
    return pop();
}

int eval(struct Node* root) {
    if (root == NULL) return 0;
    if (isdigit(root->data)) return root->data - '0';
    int left = eval(root->left);
    int right = eval(root->right);
    switch (root->data) {
        case '+': return left + right;
        case '-': return left - right;
        case '*': return left * right;
        case '/': return left / right;
    }
    return 0;
}

int main() {
    char postfix[] = "23*54*+";
    struct Node* root = constructTree(postfix);

    printf("Result: %d", eval(root));
    return 0;
}
```

2.3 Huffman Encoding Tree

Huffman Coding is a lossless data compression algorithm that assigns shorter binary codes to frequent characters and longer codes to rare ones.

Construction of Huffman Tree

```
#include <stdio.h>
#include <stdlib.h>
```

```

struct Node {
    char ch;
    int freq;
    struct Node *left, *right;
};

struct Node* createNode(char ch, int freq) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->ch = ch;
    node->freq = freq;
    node->left = node->right = NULL;
    return node;
}

struct Node* extractMin(struct Node* heap[], int *size) {
    int min = 0;
    for (int i = 1; i < *size; i++)
        if (heap[i]->freq < heap[min]->freq) min = i;
    struct Node* temp = heap[min];
    heap[min] = heap[--(*size)];
    return temp;
}

struct Node* buildHuffmanTree(char ch[], int freq[], int n) {
    struct Node* heap[100];
    for (int i = 0; i < n; i++)
        heap[i] = createNode(ch[i], freq[i]);

    int size = n;
    while (size > 1) {
        struct Node* left = extractMin(heap, &size);
        struct Node* right = extractMin(heap, &size);
        struct Node* top = createNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;
        heap[size++] = top;
    }
    return heap[0];
}

void printCodes(struct Node* root, int arr[], int top) {
    if (root->left) {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1);
    }
    if (root->right) {
        arr[top] = 1;
        printCodes(root->right, arr, top + 1);
    }
    if (!root->left && !root->right) {
        printf("%c: ", root->ch);
        for (int i = 0; i < top; i++) printf("%d", arr[i]);
        printf("\n");
    }
}

int main() {
    char ch[] = {'A', 'B', 'C', 'D', 'E'};
    int freq[] = {5, 9, 12, 13, 16};
    int n = 5;

    struct Node* root = buildHuffmanTree(ch, freq, n);
    int arr[100], top = 0;

    printf("Huffman Codes:\n");
    printCodes(root, arr, top);
    return 0;
}

```